

Build
Learn
Practice
Grow !

"Fast APIs for a Faster World"

@codedsoul_05

Better
Developers
Brighter
Future !



FastAPI

From
Basics to
Production

Cheat Sheet

&

100 Interview Questions

- ✓ Fast
- ✓ Modern
- ✓ Easy to Use
- ✓ Type Hints
- ✓ Production Ready

Why
FastAPI ?

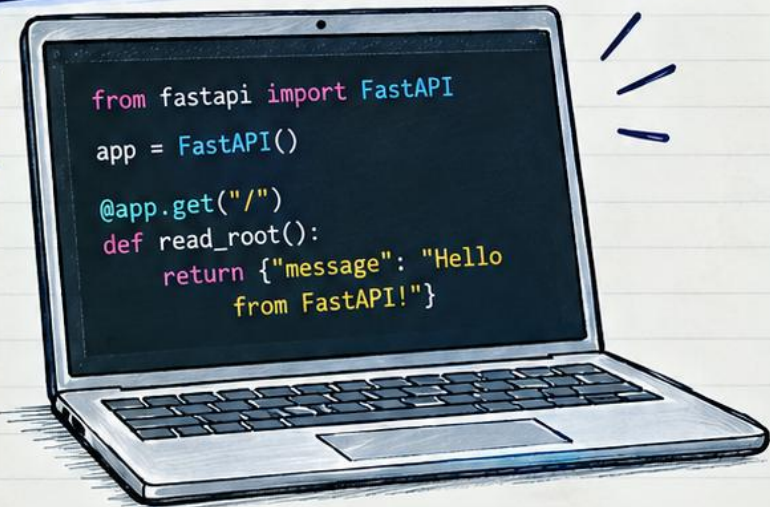
Clean
Architecture

Scalable APIs

Testing &
Deployment

Interview
Preparation

Build
Real World
Projects !



Python



FastAPI



API
Development



Databases
(SQLAlchemy)



Docker



Deployment



Testing
(Pytest)

Same
Code
Bigger
Opportunities !

Code Today Better Tomorrow !

@codedsoul_05

1. FastAPI Introduction + Why FastAPI ?

@codedsoul_05
Build
Modern
APIs !

1. What is FastAPI ?

- FastAPI is a modern, high performance Python web framework for building APIs.
- It is built on top of Starlette (for web handling) and Pydantic (for data validation).
- It allows you to build robust, production ready APIs with less code and more features.
- It supports type hints, automatic documentation and async programming.
- It is designed to be fast, easy to learn and developer friendly.

2. Why FastAPI ?

- High performance (comparable to Node.js and Go)
- Easy to learn and use
- Automatic interactive API documentation
- Built in data validation (Pydantic)
- Supports async and await
- Type hints and modern Python features
- Modular and flexible
- Production ready (used by big companies)
- Excellent community support

"Less code. More features. Higher performance."

3. Simple Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from FastAPI!"}
```

- FastAPI application object is created.
- We define a route (/) using the @app.get decorator.
- When you visit `http://127.0.0.1:8000/` you will see a JSON response.

4. Technology Stack

- FastAPI** → Main framework (built on Starlette)
- Starlette** → Handles the web layer (routing, requests, responses)
- Pydantic** → Data validation using Python type hints
- Uvicorn** → ASGI server to run the application

5. FastAPI vs Traditional Frameworks

Feature	FastAPI	Flask	Django
Performance	Very High	Moderate	Moderate
Data Validation	Automatic (Pydantic)	Manual	Manual
Documentation	Auto Generated	Needs Extension	Needs Setup
Type Hints	Built in	Not by default	Partial
Async Support	Yes	Limited	Partial
Learning Curve	Easy	Easy	Steeper
Use Case	APIs	Small / Medium Projects	Large Projects

"FastAPI gives you modern features with minimal effort."

6. Real World Use Cases

- Building REST APIs and microservices
- Machine learning model deployment
- Real time applications (WebSockets)
- Internal tools and dashboards
- Mobile and frontend backends
- Serverless functions
- Any application where you need high performance APIs with modern Python features

7. Key Takeaways

- FastAPI is a modern and fast Python framework.
- It is built on Starlette and Pydantic.
- It provides automatic documentation and data validation.
- It is easy to learn and used in real world applications.

2. Installation + Project Setup

@codedsoul_05

Set up once.
Build anything
with FastAPI!

● 1. Prerequisites

- Basic knowledge of Python
- Python 3.8 or later (recommended 3.10+)
- pip (Python package manager)
- Code editor (VS Code recommended)
- Terminal / Command Prompt
- Git (optional)

● 2. Install FastAPI

```
pip install fastapi
```

→ Installs FastAPI

```
pip install "uvicorn[standard]"
```

→ Installs Uvicorn (ASGI server)
with standard dependencies

● 3. Verify Installation

```
python --version  
pip --version  
fastapi --version  
uvicorn --version
```

→ Check Python version
→ Check pip version
→ Check FastAPI version
→ Check Uvicorn version

Expected Output (example)

```
Python 3.10.12  
pip 23.2.1  
fastapi 0.110.0  
uvicorn 0.29.0
```

● 4. Create a New Project

```
mkdir fastapi-app
```

→ Create project folder

```
cd fastapi-app
```

→ Move into the project folder

● 5. Create a Virtual Environment (Recommended)

```
python -m venv .env  
.env\Scripts\activate # Windows  
source .env/bin/activate # macOS/Linux
```

→ Create virtual environment
→ Activate (Windows)
→ Activate (macOS/Linux)

After activation

You will see (.env) in your terminal, which means the environment is active.

● 6. Install Dependencies (again inside venv)

```
pip install fastapi  
pip install "uvicorn[standard]"
```

→ Install FastAPI
→ Install Uvicorn

● 7. Create the Project Structure

```
fastapi-app/  
├── app/  
│   ├── main.py  
│   └── __init__.py  
├── .env/  
├── requirements.txt (optional)  
└── README.md (optional)
```

→ Project root folder
→ Application code folder
→ Main application file
→ Makes app a Python package
→ Virtual environment folder
→ List of dependencies
→ Project documentation

● 8. Create main.py

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/")  
def root():  
    return {"message": "Hello from FastAPI!"}
```

● 9. Run the Development Server

```
uvicorn app.main:app --reload
```

→ Starts the development server
→ Open <http://127.0.0.1:8000> in your browser

You should see:

```
{  
  "message": "Hello from FastAPI!"  
}
```

Organize
Build
Scale!

3. Project Structure + Application Instance

@codedsoul_05

Clean
Structure
Better
Projects!

1. Recommended Project Structure

```
fastapi-app/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── routers/
│   │   ├── __init__.py
│   │   └── users.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── user.py
│   ├── schemas/
│   │   ├── __init__.py
│   │   └── user.py
│   ├── services/
│   │   ├── __init__.py
│   │   └── user_service.py
│   └── core/
│       ├── __init__.py
│       └── config.py
├── .venv/
├── requirements.txt
└── README.md
```

→ # Project root folder
→ # Application package
→ # Marks it as a Python package
→ # FastAPI app instance
→ # API route files
→ # User related routes
→ # Database models (SQLAlchemy)
→ # User model
→ # Pydantic schemas (request/response)
→ # User schema
→ # Business logic / services
→ # User service functions
→ # Configuration files
→ # App settings (env variables, etc.)
→ # Virtual environment
→ # Project dependencies
→ # Project documentation

You can keep it simple in the beginning and add more folders as your project grows.

3. Application Instance (main.py)

```
from fastapi import FastAPI

app = FastAPI(
    title="My FastAPI App",
    description="A simple and scalable FastAPI project",
    version="1.0.0"
)

@app.get("/")
def read_root():
    return {"message": "Hello from FastAPI!"}
```

This is the main application instance. It creates a FastAPI app and defines a simple root route.

5. Check API Documentation

FastAPI automatically generates interactive docs:

- Swagger UI → <http://127.0.0.1:8000/docs>
- ReDoc → <http://127.0.0.1:8000/redoc>

No extra configuration needed.
FastAPI provides automatic documentation.

2. Create Project Structure (Commands)

```
mkdir fastapi-app
cd fastapi-app
python -m venv .venv

# Activate environment
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

# Create folders
mkdir app app/routers app/models app/schemas \
app/services app/core

# Create files
type nul > app/__init__.py # Windows
touch app/__init__.py      # macOS/Linux

# (or create files manually in VS Code)
```

Set up once,
Use for
every project!

4. Run the Project

```
uvicorn app.main:app --reload
```

- app.main → path to your main.py file
- app → FastAPI application instance
- --reload → auto reload on code changes (useful in development)

After running, open <http://127.0.0.1:8000> in your browser. You will see the API response.

6. Next Steps

- ✓ Create routers (app/routers/)
- ✓ Add models (app/models/)
- ✓ Define schemas (app/schemas/)
- ✓ Add services (app/services/)
- ✓ Use environment variables (app/core/config.py)
- ✓ Connect database (in next pages)
- ✓ Build more API endpoints

"A clean
structure today
saves hours
tomorrow."

Code
Structure
Progress!

@codedsoul_05

Small Steps
Big Projects!

4. Path Operations + HTTP Methods

"Path operations define what happens when a client sends a request to a specific URL."

1. What are Path Operations?

- A path operation is a function that handles an HTTP request for a specific path (URL).
- It tells FastAPI what to do when a user accesses a particular endpoint.
- You define path operations using decorators like `@app.get()`, `@app.post()`, etc.
- Each path operation can have a path, HTTP method, parameters, request body and response.

2. Basic Example

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from FastAPI!"}

@app.get("/about")
def about():
    return {"message": "This is About page"}
```

Each function is a path operation.
`@app.get("/")` means a GET request to root path (/)

3. Common HTTP Methods

Method	Purpose	Use Case	Idempotent
GET	Read data	Fetch resources	Yes
POST	Create data	Create a new resource	No
PUT	Update data (full)	Replace entire resource	Yes
PATCH	Update data (partial)	Update specific fields	No
DELETE	Delete data	Remove a resource	Yes

FastAPI supports all standard HTTP methods.

5. Run and Test

- 1 Run the server

```
uvicorn main:app --reload
```
- 2 Open in browser

```
http://127.0.0.1:8000/docs
```
- 3 Test the endpoints using Swagger UI

FastAPI provides an interactive documentation (Swagger UI) where you can test all your endpoints directly.

6. Request and Response Flow



4. Example with All HTTP Methods

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/items")
def get_items():
    return {"message": "Get all items"}

@app.post("/items")
def create_item():
    return {"message": "Create an item"}

@app.put("/items/1")
def update_item():
    return {"message": "Update item 1"}

@app.patch("/items/1")
def partial_update_item():
    return {"message": "Partially update item 1"}

@app.delete("/items/1")
def delete_item():
    return {"message": "Delete item 1"}
```

Same path can have different HTTP methods!

7. Key Takeaways

- ✓ Path operations handle HTTP requests.
- ✓ Use decorators (`@app.get()`, `@app.post()`, etc.).
- ✓ FastAPI supports all standard HTTP methods.
- ✓ Each path can have only one function per method.
- ✓ Test your APIs using the interactive docs.
- ✓ Keep your endpoints clean and meaningful.

5. Path Parameters + Query Parameters

1. What are Path Parameters ?

- Path parameters are part of the URL path.
- They are used to identify a specific resource.
- They are defined using curly braces { } in the path.
- They are required (must be provided).

Example:

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id, "message": "User details"}
```

Here user_id is a path parameter.
Example URL:
/users/101

3. Multiple Path Parameters

```
@app.get("/users/{user_id}/posts/{post_id}")
def get_post(user_id: int, post_id: int):
    return {
        "user_id": user_id,
        "post_id": post_id,
        "message": "Post details"
    }
```

You can use multiple path parameters in a single route.

5. Path Parameter with Data Types

```
@app.get("/products/{product_id}")
def get_product(product_id: int):
    return {"product_id": product_id}

@app.get("/products/{name}")
def get_product_by_name(name: str):
    return {"name": name}
```

You can define data types like int, str, float, bool, UUID, etc. FastAPI will automatically validate the type.

8. Combining Path and Query Parameters

```
@app.get("/users/{user_id}")
def get_user(user_id: int, details: bool = False):
    return {"user_id": user_id, "details": details}
```

Here user_id is a path parameter and details is a query parameter.
Example URL: /users/101?details=true

2. What are Query Parameters ?

- Query parameters are additional parameters passed in the URL after a ?.
- They are used to filter, search, or modify the response.
- They are optional (unless explicitly required).
- They are key-value pairs.

Example:

```
@app.get("/items")
def get_items(skip: int = 0, limit: int = 10):
    return {
        "skip": skip,
        "limit": limit,
        "message": "List of items"
    }
```

Here skip and limit are query parameters.
Example URL:
/items?skip=0&limit=10

4. Multiple Query Parameters

```
@app.get("/search")
def search(q: str, category: str = None, limit: int = 10):
    return {
        "query": q,
        "category": category,
        "limit": limit
    }
```

You can pass multiple query parameters.
Example URL:
/search?q=phone&category=tech&limit=5

6. Optional Query Parameters

```
from typing import Optional

@app.get("/items")
def get_items(category: Optional[str] = None):
    return {"category": category}
```

Use Optional to make query parameters optional.

7. Required Query Parameter

```
@app.get("/search")
def search(q: str):
    return {"query": q}
```

If no default value is given, query parameter becomes required.

9. Key Takeaways

- ✓ Path parameters are part of the URL and required.
- ✓ Query parameters are optional and passed after ?.
- ✓ You can use multiple path and query parameters.
- ✓ FastAPI automatically handles type validation.
- ✓ You can define default values for query parameters.
- ✓ Path and query parameters can be used together.
- ✓ Useful for building flexible and powerful APIs.

Define
Validate
Parse
Real World Ready!

6. Request Body + Pydantic Models

@coded soul_05
FastAPI
Pydantic
Better APIs!

1. What is a Request Body?

- A request body is the data sent by the client in the HTTP request (usually in POST, PUT, PATCH).
- It is used to send complex data like JSON, instead of just URL parameters.
- In FastAPI, we use Pydantic models to define and validate the request body.

2. What is Pydantic?

- Pydantic is a data validation library used by FastAPI.
- It allows you to define models with type hints.
- It automatically validates and converts the incoming data.
- If data is invalid, FastAPI returns a clear error response (422 Unprocessable Entity).

Pydantic =
Data Validation
+ Type Hints
+ Automatic Parsing

3. Simple Request Body Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int
    email: str
```

```
@app.post("/users")
def create_user(user: User):
    return {"message": "User created", "user": user}
```

Here, user is a
Pydantic model.
FastAPI will
automatically
validate the
request body.

@coded soul_05

4. Example Request (JSON)

```
POST /users
Content-Type: application/json

{
  "name": "Nikhil",
  "age": 24,
  "email": "nikhil@example.com"
}
```

```
Response (Success)

{
  "message": "User created",
  "user": {
    "name": "Nikhil",
    "age": 24,
    "email": "nikhil@example.com"
  }
}
```

5. Pydantic Model with Validation

```
from pydantic import BaseModel, EmailStr, Field

class User(BaseModel):
    name: str = Field(..., min_length=3, max_length=50)
    age: int = Field(..., gt=0, lt=120)
    email: EmailStr # Validates email format
    is_active: bool = True
```

- Field() allows you to add extra validation rules.
- EmailStr ensures a valid email format.
- gt = greater than, lt = less than.
- You can also set default values.

6. Nested Models

```
from pydantic import BaseModel

class Address(BaseModel):
    city: str
    pincode: int

class User(BaseModel):
    name: str
    age: int
    address: Address # Nested model
```

Pydantic also
supports nested
models for complex
data structures.

@coded soul_05

7. List of Items in Request Body

```
from typing import List

class Item(BaseModel):
    name: str
    price: float

@app.post("/items")
def create_items(items: List[Item]):
    return {"message": "Items received", "items": items}
```

You can receive a
list of objects in
the request body
using List[Model].

@coded soul_05

8. Key Takeaways

- ✓ Use Pydantic models to define request body.
- ✓ Pydantic automatically validates and parses data.
- ✓ You can add custom validation using Field().
- ✓ Supports nested models and lists.
- ✓ Returns clear error messages for invalid data.
- ✓ Makes your API more robust and performant.

Clean Code
Strong Backend!

Build
Learn
Grow!

7. Response Models + Status Codes

1. What are Response Models ?

- Response models define the structure of the data returned by the API.
- They help in data validation, serialization and automatic documentation (Swagger UI).
- Even if your function returns extra fields, FastAPI will filter the response based on the model.
- We use Pydantic models for response models.

2. Basic Response Model Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UserResponse(BaseModel):
    id: int
    name: str
    email: str

@app.get("/users/{user_id}", response_model=UserResponse)
def get_user(user_id: int):
    return {
        "id": user_id,
        "name": "Nikhil",
        "email": "nikhil@example.com",
        "extra_field": "this will be ignored" # not in model
    }
```

This model defines the structure of the response.

@codedsoul_05

3. Response Model with a List

```
from typing import List

class UserResponse(BaseModel):
    id: int
    name: str
    email: str

@app.get("/users", response_model=List[UserResponse])
def get_users():
    return [
        {"id": 1, "name": "Nikhil", "email": "nikhil@example.com"},
        {"id": 2, "name": "Payal", "email": "payal@example.com"}
    ]
```

You can also use List[Model] for multiple responses.

4. Using status_code Parameter

```
from fastapi import status

@app.post("/users", response_model=UserResponse,
          status_code=status.HTTP_201_CREATED)
def create_user():
    return {
        "id": 1,
        "name": "Nikhil",
        "email": "nikhil@example.com"
    }
```

You can set the status code for successful responses.

5. Common HTTP Status Codes

Status Code	Constant (fastapi.status)	Meaning
200	HTTP_200_OK	Successful GET request
201	HTTP_201_CREATED	Resource created successfully
204	HTTP_204_NO_CONTENT	Successful request, no content
400	HTTP_400_BAD_REQUEST	Invalid request
401	HTTP_401_UNAUTHORIZED	Authentication required
403	HTTP_403_FORBIDDEN	Access denied
404	HTTP_404_NOT_FOUND	Resource not found
422	HTTP_422_UNPROCESSABLE_ENTITY	Validation error
500	HTTP_500_INTERNAL_SERVER_ERROR	Server error

6. Custom Response with JSONResponse

```
from fastapi.responses import JSONResponse

@app.get("/error")
def custom_error():
    return JSONResponse(
        status_code=404,
        content={
            "error": "User not found",
            "message": "The requested user does not exist"
        }
    )
```

Use JSONResponse when you want to return a custom response format.

@codedsoul_05

7. Key Takeaways

- ✓ Use response_model to define the response structure.
- ✓ FastAPI filters extra fields not present in the model.
- ✓ Use status_code to set the appropriate HTTP status code.
- ✓ Use pydantic models for both request and response.
- ✓ Common status codes: 200, 201, 400, 401, 403, 404, 422, 500.
- ✓ Better responses make your API more consistent and professional.

Clean
Data
Reliable APIs!

8. Pydantic Validation + Schemas

@codedsoul_05

Better
Schemas
Better
Backends!

"Pydantic helps you validate, parse and manage data using Python type hints."

1. Why Pydantic for Validation?

- Automatically validates incoming data.
- Uses Python type hints (int, str, etc.).
- Provides clear error messages.
- Converts and parses data to correct types.
- Helps in request and response models.
- Used by FastAPI internally.

Less bugs
More reliable
APIs!

2. Basic Pydantic Model (Schema)

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
    email: str
```

This is a Pydantic
model (schema).
It defines the
structure and
data types.

3. Using Schema in FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int
    email: str
```

```
@app.post("/users")
def create_user(user: User):
    return {"message": "User created", "user": user}
```

FastAPI
automatically
validates the
request body
using this schema.

4. Validation in Action

Invalid Request (age is string)

```
POST /users
{
  "name": "Nikhil",
  "age": "twenty",
  "email": "nikhil@example.com"
}
```

Error Response (422)

```
{
  "detail": [
    {
      "loc": ["body", "age"],
      "msg": "Input should be  
a valid integer",
      "type": "int_parsing"
    }
  ]
}
```

FastAPI + Pydantic automatically returns a 422 error
for invalid data.

Validate
Early
Save Time!

5. Advanced Validation Examples

```
from pydantic import BaseModel, EmailStr, Field, conint

class User(BaseModel):
    name: str = Field(..., min_length=3, max_length=50)
    age: conint(ge=0, le=120) # between 0 and 120
    email: EmailStr # must be a valid email
    is_active: bool = True
    bio: str | None = Field(None, max_length=200)
```

- Field() is used to add extra validation rules.
- EmailStr ensures a valid email format.
- conint() adds numeric constraints (greater than, less than).
- You can also use constr(), conlist(), confloat(), etc.

6. Schema for Response (Separate Model)

```
class UserResponse(BaseModel):
    id: int
    name: str
    age: int
    email: EmailStr
    is_active: bool
```

You can create different
schemas for request and
response. This helps
in hiding sensitive fields
and returning only
required data.

7. Nested Schemas

```
class Address(BaseModel):
    city: str
    pincode: int

class User(BaseModel):
    name: str
    age: int
    address: Address # Nested model
```

Pydantic also
supports nested
models for complex
data structures.

8. Key Takeaways

- ✓ Use Pydantic models (schemas) for data
- ✓ FastAPI automatically validates request validation.
- ✓ Use Field() for custom validation (min, max, etc.).
- ✓ Use built-in types like EmailStr, conint(), constr().
- ✓ You can create separate schemas for request and response.
- ✓ Supports nested models.
- ✓ Returns clear error messages for invalid data.
- ✓ Makes your API more robust, secure and maintainable.

@codedsoul_05

Small
Schemas
Big Impact!

Reusable
Logic
Cleaner Code
Better APIs!

9. Dependency Injection + Depends()

@coded soul_05

Same
Logic
Multiple Places
More Power!

"Dependency Injection allows you to provide and reuse common logic easily in FastAPI using Depends()."

1. What is Dependency Injection ?

- Dependency Injection (DI) is a design pattern where dependencies (like database, authentication, etc.) are provided from outside instead of creating them inside your route functions.
- In FastAPI, we use Depends() to declare dependencies.
- It helps in code reusability, clean code and testing.
- FastAPI automatically handles the dependency resolution for you.

2. Basic Example of Depends()

```
from fastapi import FastAPI, Depends
app = FastAPI()
```

```
def get_current_user():
    return {"name": "Nikhil", "role": "user"}
```

```
@app.get("/profile")
def read_profile(user: dict = Depends(get_current_user)):
    return {"message": "User profile", "user": user}
```

Depends() tells FastAPI to call the function and pass its return value.

3. Using Depends() with Parameters

```
from fastapi import FastAPI, Depends, HTTPException
app = FastAPI()
```

```
def verify_token(token: str):
    if token != "secret":
        raise HTTPException(status_code=401, detail="Invalid token")
    return {"token": token}
```

```
@app.get("/dashboard")
def dashboard(data: dict = Depends(verify_token)):
    return {"message": "Welcome!", "data": data}
```

You can also pass parameters to dependency functions.

4. Class Based Dependencies

```
class Settings:
    def __call__(self):
        return {"app_name": "My FastAPI App", "debug": True}

settings = Settings()

@app.get("/settings")
def get_settings(config: dict = Depends(settings)):
    return config
```

Classes with __call__() can also be used as dependencies.

5. Real World Example (Database Dependency)

```
from fastapi import Depends, FastAPI
from sqlalchemy.orm import Session
from database import SessionLocal # your DB setup

app = FastAPI()
```

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

```
@app.get("/items")
def read_items(db: Session = Depends(get_db)):
    return {"message": "Fetch items from database"}
```

Commonly used to provide a database session in routes.

6. Multiple Dependencies

```
from fastapi import Depends
```

```
def get_current_user():
    return {"name": "Nikhil"}
```

```
def get_db():
    return "DB Session"
```

```
@app.get("/data")
def get_data(user: dict = Depends(get_current_user),
             db: str = Depends(get_db)):
    return {"user": user, "db": db}
```

You can use multiple dependencies in a single route.

7. Built-in Dependencies

```
from fastapi import Query, Path, Depends
```

```
@app.get("/users/{user_id}")
def get_user(
    user_id: int = Path(..., description="User ID"),
    active: bool = Query(True, description="Is active user?"),
    user: dict = Depends(get_current_user)
):
    return {"user_id": user_id, "active": active, "user": user}
```

You can combine Depends() with Path, Query and other parameters.

8. Key Takeaways

- ✓ Depends() is used for dependency injection in FastAPI.
- ✓ Helps in reusing common logic (auth, DB, config, etc.).
- ✓ Supports both simple functions and class based dependencies.
- ✓ Can be combined with Path, Query and other parameters.
- ✓ Supports multiple dependencies in a single route.
- ✓ Makes your code modular, clean and testable.

Write Once
Use Everywhere!

Clean
Modular
Maintainable!

More
Ways to
Handle Data!
More Powerful APIs!

10. Headers, Cookies + Form Data

@codedsoul_05

Build
Flexible
Real World
APIs!

"FastAPI gives easy access to headers, cookies and form data using simple dependencies."

1. Request Headers

- Headers contain metadata about the request.
- In FastAPI, use `Header()` from `fastapi` to access headers.
- You can define default values, make them optional or convert to a specific type.

```
from fastapi import FastAPI, Header
app = FastAPI()

@app.get("/headers")
def get_headers(
    user_agent: str = Header(None),
    x_token: str = Header(None)
):
    return {
        "user_agent": user_agent,
        "x_token": x_token
    }
```

`Header()` helps
you get values
from request
headers.

Example Request
Headers:
User-Agent: Chrome
X-Token: abc123

2. Optional and Required Headers

```
from fastapi import Header
@app.get("/items")
def read_items(
    x_token: str = Header(...), # required
    user_agent: str = Header(None) # optional
):
    return {
        "token": x_token,
        "user_agent": user_agent
    }
```

Use ... to make
header required.

Use None to
make header
optional.

3. Cookies

- Cookies are key-value pairs stored in the user's browser.
- In FastAPI, use `Cookie()` to read cookies from the request.
- You can also set cookies in the response using `Response`.

```
from fastapi import Cookie, Response
@app.get("/cookies")
def read_cookies(
    session_id: str = Cookie(None)
):
    return {"session_id": session_id}

@app.get("/set-cookie")
def set_cookie(response: Response):
    response.set_cookie(key="session_id", value="abc123",
                        httponly=True, max_age=3600)
    return {"message": "Cookie set successfully"}
```

`Cookie()` helps you
read cookies from
the request.
Use `Response.set_cookie()`
to set cookies.

4. Form Data

- Form data is used when submitting data from HTML forms (`application/x-www-form-urlencoded` or `multipart/form-data`).
- In FastAPI, use `Form()` to receive form data.
- Commonly used for login forms, file uploads etc.

```
from fastapi import FastAPI, Form
app = FastAPI()

@app.post("/login")
def login(
    username: str = Form(...),
    password: str = Form(...)
):
    return {
        "message": "Login successful",
        "username": username
    }
```

`Form()` is used to
extract form fields
from the request
body.

Content-Type for
Form Data:
`application/x-www-
form-urlencoded`

5. File Upload with Form Data (Multipart)

- When uploading files, use `UploadFile` and `File()` along with `Form()` for other fields.
- Form data with files uses `multipart/form-data`.

```
from fastapi import FastAPI, File, UploadFile, Form
app = FastAPI()

@app.post("/upload")
async def upload_file(
    name: str = Form(...),
    file: UploadFile = File(...)
):
    return {
        "filename": file.filename,
        "name": name,
        "message": "File uploaded successfully"
    }
```

Use `File()` for files
and `Form()` for
other form fields.

Content-Type for
File Upload:
`multipart/form-data`

6. Key Takeaways

- ✓ Use `Header()` to access request headers.
- ✓ Use `Cookie()` to read cookies and `Response.set_cookie()` to set cookies.
- ✓ Use `Form()` to handle form data.
- ✓ Use `File()` and `UploadFile()` for file uploads.
- ✓ You can make headers, cookies and form fields optional or required.
- ✓ These features help you build real world, production ready APIs.
- ✓ FastAPI makes handling request data simple and clean.

Learn
Build
Implement!

@codedsoul_05

Same
Code
Bigger Possibilities!

Upload
Serve
Build Better
FastAPI APIs!

11. File Uploads + Static Files

@codedsoul_05

Simple
Features
Big Possibilities!

"FastAPI makes it easy to handle file uploads and serve static files for your application."

1. File Uploads - Basics

- FastAPI provides `File()` and `UploadFile` for handling file uploads.
- `UploadFile` is recommended (better performance for large files).
- You can accept single or multiple files.
- Uploaded files can be read, saved or processed (image, PDF, etc.).

2. Single File Upload Example

```
from fastapi import FastAPI, File, UploadFile
from typing import Annotated
```

```
app = FastAPI()
```

```
@app.post("/upload")
async def upload_file(file: Annotated[UploadFile, File()]):
    contents = await file.read()
    with open(f"uploads/{file.filename}", "wb") as f:
        f.write(contents)
    return {
        "filename": file.filename,
        "content_type": file.content_type,
        "message": "File uploaded successfully"
    }
```

`UploadFile` gives you file metadata like filename, content_type, etc.

3. Multiple File Upload Example

```
from fastapi import FastAPI, File, UploadFile
from typing import List
```

```
app = FastAPI()
```

```
@app.post("/upload-multiple")
async def upload_multiple_files(files: List[UploadFile] = File(...)):
    filenames = []
    for file in files:
        contents = await file.read()
        with open(f"uploads/{file.filename}", "wb") as f:
            f.write(contents)
        filenames.append(file.filename)
    return {
        "filenames": filenames,
        "message": "All files uploaded successfully"
    }
```

You can accept multiple files using `List[UploadFile]` with `File(...)`.

4. Save Uploaded Files in a Specific Folder

```
import os
os.makedirs("uploads", exist_ok=True)
with open(f"uploads/{file.filename}", "wb") as f:
    f.write(await file.read())
```

Always create the directory if it doesn't exist.

5. Serve Static Files

- Static files like CSS, JavaScript, images can be served using `StaticFiles`.
- This is useful for frontend files, images or other assets.
- You need to mount a directory to a route.

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()
app.mount("/static", StaticFiles(directory="static"),
        name="static")
```

All files inside the "static" folder will be accessible at `/static`.

6. Example Folder Structure

```
my_project/
├── main.py
├── static/
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── app.js
│   └── images/
│       └── logo.png
└── uploads/ # uploaded files will be saved here
```

You can organize your static files like CSS, JS, images in a folder and serve them easily.

7. Accessing Static Files

- If you place a file `static/images/logo.png`.
- it will be accessible at `http://localhost:8000/static/images/logo.png`
- You can use these static files in your frontend (HTML, CSS, JS).

```
<html>
<head>
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  
  <script src="/static/js/app.js"></script>
</body>
</html>
```

Use the `/static` path in your HTML to access files.

8. Key Takeaways

- ✓ Use `UploadFile` and `File()` for handling file uploads.
- ✓ You can accept single or multiple files.
- ✓ Save uploaded files securely (create directory if needed).
- ✓ Use `StaticFiles` to serve static files (CSS, JS, images, etc.).
- ✓ Static files are accessible via the mounted route (e.g. `/static`).
- ✓ Useful for building full stack applications with FastAPI.
- ✓ Simple, powerful and easy to implement.

@codedsoul_05

Code
Upload
Deploy
Repeat!

Same
Code
Bigger Dreams!

12. Authentication + OAuth2 + JWT

@coded soul_05

Auth
Today
Safer APIs
Tomorrow!

"Authentication ensures who you are. Authorization decides what you can do."

1. Why Authentication ?

- Protects your API from unauthorized access.
- Ensures only valid users can access protected routes.
- JWT (JSON Web Token) is a common way to handle authentication in APIs.
- OAuth2 provides a standard way for authentication and authorization.
- FastAPI provides easy support using OAuth2PasswordBearer.

2. Install Required Packages

```
pip install fastapi[all] python-jose[cryptography]  
passlib[bcrypt]
```

python-jose for
JWT
passlib for
password hashing

3. User Model (Pydantic + Fake DB)

```
from pydantic import BaseModel  
from typing import Optional  
  
class User(BaseModel):  
    username: str  
    disabled: Optional[bool] = None
```

```
# Fake user database  
fake_users_db = {  
    "nikhil": {  
        "username": "nikhil",  
        "hashed_password": "$2b$12$examplehashedpassword"  
    }  
}
```

In real projects,
use a proper
database instead
of a fake DB.

@coded soul_05

4. Password Hashing (using passlib)

```
from passlib.context import CryptContext  
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")  
  
def verify_password(plain_password, hashed_password):  
    return pwd_context.verify(plain_password, hashed_password)  
  
def get_password_hash(password):  
    return pwd_context.hash(password)
```

Always hash passwords.
Never store plain text
passwords !

5. Create JWT Token

```
from jose import JWTError, jwt  
from datetime import datetime, timedelta  
  
SECRET_KEY = "your-secret-key" # use strong key in production  
ALGORITHM = "HS256"  
ACCESS_TOKEN_EXPIRE_MINUTES = 30  
  
def create_access_token(data: dict):  
    to_encode = data.copy()  
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)  
    to_encode.update({"exp": expire})  
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

JWT contains user
information and expiry time.
It is signed using a secret key.

6. OAuth2 Scheme

```
from fastapi.security import OAuth2PasswordBearer  
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
```

tokenUrl points
to the endpoint
that provides
the JWT token.

7. Login Endpoint (Get Token)

```
from fastapi import FastAPI, Depends, HTTPException, status  
from fastapi.security import OAuth2PasswordRequestForm  
  
app = FastAPI()  
  
@app.post("/token")  
async def login(form_data: OAuth2PasswordRequestForm = Depends()):  
    user = fake_users_db.get(form_data.username)  
    if not user or not verify_password(form_data.password, user["hashed_password"]):  
        raise HTTPException(  
            status_code=status.HTTP_401_UNAUTHORIZED,  
            detail="Incorrect username or password"  
        )  
    access_token = create_access_token(data={"sub": user["username"]})  
    return {"access_token": access_token, "token_type": "bearer"}
```

OAuth2PasswordRequestForm
reads username and password
from form data.

@coded soul_05

8. Protecting Routes

```
from fastapi import Depends  
from jose import JWTError, jwt  
  
async def get_current_user(token: str = Depends(oauth2_scheme)):  
    credentials_exception = HTTPException(  
        status_code=status.HTTP_401_UNAUTHORIZED,  
        detail="Could not validate credentials"  
    )  
    try:  
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])  
        username: str = payload.get("sub")  
        if username is None:  
            raise credentials_exception  
        user = fake_users_db.get(username)  
        if user is None:  
            raise credentials_exception  
        return User(username=username)  
    except JWTError:  
        raise credentials_exception
```

Extract the token,
validate it and
get the current user.

This route is protected
and requires a valid token.

```
@app.get("/users/me")  
async def read_users_me(current_user: User = Depends(get_current_user)):  
    return {"message": f"Hello {current_user.username}"}
```

9. How It Works (Flow)



10. Key Takeaways

- ✓ Use OAuth2PasswordBearer for handling authentication.
- ✓ Use JWT for stateless authentication.
- ✓ Always hash passwords using passlib.
- ✓ Set a strong secret key and short token expiry time.
- ✓ In production, use a real database and proper user management.
- ✓ Authentication and authorization are essential for building secure APIs.

Secure APIs
Build Trust
Create Better
Products !

Extend
Your API
More Flexible
More Secure!

13. Middleware + CORS

@codedsoul_05

Same
Code
Bigger
Possibilities!

"Middleware lets you run logic before and after each request, and CORS helps your API accept requests from different origins (like your frontend)."

1. What is Middleware ?

- Middleware is code that runs before and after the request reaches your route.
- It can be used for logging, authentication, timing, modifying request/response, etc.
- In FastAPI, you can create custom middleware or use built-in middleware classes.

2. Simple Middleware Example

```
from fastapi import FastAPI, Request
app = FastAPI()
```

```
@app.middleware("http")
async def log_requests(request: Request, call_next):
    print(f"Request: {request.method} {request.url}")
    response = await call_next(request)
    print(f"Response Status: {response.status_code}")
    return response
```

This middleware
logs every request
and response.

```
@app.get("/")
def home():
    return {"message": "Hello from FastAPI"}
```

3. Class Based Middleware

```
from fastapi import FastAPI, Request
from starlette.middleware.base import BaseHTTPMiddleware
app = FastAPI()
```

```
class TimingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request,
                       start_time = time.time()):
        response = await call_next(request)
        process_time = time.time() - start_time
        print(f"Process Time: {process_time:.4f} seconds")
        response.headers["X-Process-Time"] = str(process_time)
        return response

app.add_middleware(TimingMiddleware)
```

You can also
create class based
middleware for
more complex logic.

4. What is CORS ?

- CORS (Cross Origin Resource Sharing) allows your API to accept requests from different domains (origins).
- Browsers block requests from a different origin by default for security reasons.
- In FastAPI, we use CORSMiddleware to handle CORS.

5. Enable CORS in FastAPI

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "https://myapp.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

This allows your
frontend (e.g. React)
to access the API.

6. CORS Parameters Explained

Parameter	Description
allow_origins	List of allowed origins (domains). Use ["*"] to allow all (not recommended in production).
allow_credentials	Allows cookies, authorization headers, etc.
allow_methods	Allowed HTTP methods (GET, POST, etc).
allow_headers	Allowed headers (e.g. Content-Type, Authorization).

7. Real World Example (React Frontend)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"], # React dev server
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["*"],
)
```

Now your React app
running on port 3000
can call this API.

8. Key Takeaways

- ✓ Middleware runs before and after each request.
- ✓ Use middleware for logging, authentication, timing, etc.
- ✓ FastAPI provides easy way to create custom middleware.
- ✓ CORS allows cross origin requests from frontend applications.
- ✓ Use CORSMiddleware to configure CORS.
- ✓ Be careful while allowing all origins ("*") in production.
- ✓ Middleware and CORS make your API more flexible and ready for real world applications.

Open
Connect
Collaborate!

@codedsoul_05

Build
Secure
Scalable APIs!

Handle
Errors Better
Build Reliable
APIs !

14. Exception Handling + Custom Errors

@codedsoul_05

Clean
APIs
Happy
Users !

"Good error handling does not just show errors, it gives meaningful responses and improves user experience."

1. Why Exception Handling ?

- Prevents your application from crashing.
- Gives meaningful error messages to the client.
- Helps in debugging and logging.
- In FastAPI, we can handle built-in exceptions or create custom exceptions.
- Improves API reliability and user experience.

2. Built-in HTTPException

```
from fastapi import FastAPI, HTTPException
app = FastAPI()

@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id <= 0:
        raise HTTPException(
            status_code=400,
            detail="Item ID must be greater than 0"
        )
    return {"item_id": item_id, "name": "Sample Item"}
```

Use HTTPException
to return errors
with proper status
code and message.

3. Common HTTP Status Codes

Status Code	Meaning
400	Bad Request (Invalid input)
401	Unauthorized (Authentication required)
403	Forbidden (You don't have permission)
404	Not Found (Resource not found)
409	Conflict (Resource already exists)
422	Unprocessable Entity (Validation error)
500	Internal Server Error (Unexpected error)

Use the
right status
code for
better
clarity.

4. Custom Exception Class

```
from fastapi import FastAPI, HTTPException
app = FastAPI()

class ItemNotFoundError(Exception):
    def __init__(self, item_id: int):
        self.item_id = item_id
        self.message = f"Item with id {item_id} not found"

@app.get("/products/{item_id}")
def get_product(item_id: int):
    if item_id != 1:
        raise ItemNotFoundError(item_id)
    return {"item_id": item_id, "name": "Product 1"}
```

Create custom
exception classes
for your application
specific errors.

5. Exception Handler for Custom Exception

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
app = FastAPI()

class ItemNotFoundError(Exception):
    def __init__(self, item_id: int):
        self.item_id = item_id
        self.message = f"Item with id {item_id} not found"

@app.exception_handler(ItemNotFoundError)
async def item_not_found_handler(request: Request, exc: ItemNotFoundError):
    return JSONResponse(
        status_code=404,
        content={
            "error": "ItemNotFound",
            "message": exc.message,
            "item_id": exc.item_id
        }
    )
```

Use @app.exception_handler
to define how your custom
exception will be handled.

6. Handling Validation Errors (Request Validation)

```
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
app = FastAPI()

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(
        status_code=422,
        content={
            "error": "ValidationError",
            "message": "Invalid input data",
            "details": exc.errors()
        }
    )
```

Handles errors when
the request body or
parameters are invalid.

7. Global Exception Handler (Catch All)

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
app = FastAPI()

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={
            "error": "InternalServerError",
            "message": "Something went wrong. Please try again later."
        }
    )
```

Catch all unexpected
errors to prevent
app crash.

8. Key Takeaways

- ✓ Use HTTPException for simple error responses.
- ✓ Create custom exception classes for application specific errors.
- ✓ Use @app.exception_handler to handle custom and global exceptions.
- ✓ Handle validation errors using RequestValidationError.
- ✓ Return meaningful error messages with proper status codes.
- ✓ Exception handling makes your API more robust and user friendly.

Better
Errors
Better
Products !

@codedsoul_05

Handle
Learn
Grow
Repeat !

15. Async Programming + Background Tasks

"Async makes your API handle more requests efficiently, and Background Tasks let you run non-critical work without blocking the response."

1. Why Async Programming ?

- Async allows your application to perform non-blocking I/O operations.
- While waiting for an operation (like database query or API call), other requests can be processed.
- It helps in handling more concurrent requests.
- In FastAPI, you can use `async` and `await` with path operations.
- Ideal for I/O bound tasks (database calls, external API requests, file operations).

2. Sync vs Async Example

Synchronous (Blocking)

```
import time
from fastapi import FastAPI
app = FastAPI()

@app.get("/sync")
def sync_endpoint():
    time.sleep(3) # blocks the thread
    return {"message": "This is a sync endpoint"}
```

In synchronous code, the server waits and cannot handle other requests.

Asynchronous (Non-blocking)

```
import asyncio
from fastapi import FastAPI
app = FastAPI()

@app.get("/async")
def async_endpoint():
    await asyncio.sleep(3) # non-blocking
    return {"message": "This is an async endpoint"}
```

In async code, other requests can be processed while waiting.

3. When to Use Async ?

- When performing I/O bound operations (database, external APIs, file read/write, network calls).
- Not useful for CPU bound tasks (use background tasks or separate workers in that case).
- Use `async` with libraries that support it (e.g. `httpx`, databases like `asyncpg`, `motor` for MongoDB).
- If a library does not support `async`, using `async` will not give any benefit.

Examples of Async Friendly Libraries:

- `httpx` (async HTTP client)
- databases (async DB queries)
- `motor` (async MongoDB client)
- `aiofiles` (async file operations)

Always use `async` versions of libraries to get real performance benefits.

4. Background Tasks in FastAPI

- Used to run small tasks in the background after sending the response.
- Useful for non critical tasks like sending emails, logging, file processing, etc.
- Implemented using `BackgroundTasks` from FastAPI.
- Runs in the same process (not for heavy or long-running tasks).

5. Simple Background Task Example

```
from fastapi import FastAPI, BackgroundTasks
import time
app = FastAPI()

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(f"{message}\n")
    time.sleep(2) # simulate a time-consuming task

@app.post("/send-notification")
def send_notification(background_tasks: BackgroundTasks):
    background_tasks.add_task(write_log, "User signed up")
    return {"message": "Notification will be processed in background"}
```

The response is sent immediately, and the task runs in the background.

6. Use Case: Send Email in Background

```
from fastapi import FastAPI, BackgroundTasks
app = FastAPI()

def send_email(to_email: str, subject: str, body: str):
    # integrate with your email service here
    print(f"Sending email to {to_email}")

@app.post("/signup")
def signup(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(
        send_email, email, "Welcome", "Thanks for signing up!"
    )
    return {"message": "User created. Welcome email will be sent."}
```

Great for emails, logs, reports, etc.

7. Limitations

- `BackgroundTasks` are not suitable for long-running or heavy CPU tasks.
- For heavy tasks, use a task queue like Celery with Redis/RabbitMQ.
- Background tasks run in the same process, so if the app restarts, pending tasks may be lost.
- Use for small, quick, non-critical jobs only.

8. Key Takeaways

- ✓ Use `async` and `await` for non-blocking I/O operations.
- ✓ `Async` helps in handling more concurrent requests.
- ✓ Use `BackgroundTasks` for small, non-critical tasks.
- ✓ For heavy or long-running tasks, use Celery or a task queue.
- ✓ Always use `async` compatible libraries for real performance gains.
- ✓ `Async` + `Background Tasks` make your FastAPI app fast, responsive and scalable.

From
APIs
to Real Data
Make it Happen!

16. Database Integration + SQLAlchemy

@codedsoul_05

Build
Better
Backend
Today!

"FastAPI + SQLAlchemy gives you a powerful and flexible way to work with databases in your application."

1. Why Database Integration ?

- Allows your API to store and retrieve persistent data.
- Used for real world applications (users, products, orders, etc.).
- SQLAlchemy is a powerful and flexible SQL toolkit and ORM (Object Relational Mapper) for Python.
- Works with multiple databases (SQLite, PostgreSQL, MySQL, etc.).
- In FastAPI, we usually use SQLAlchemy with a database session dependency.

2. Install Required Packages

```
pip install sqlalchemy
pip install databases # (optional, for async support)
pip install asyncpg # (for PostgreSQL async)
pip install psycopg2 # (for PostgreSQL sync)
pip install sqlite3 # (usually comes preinstalled)
```

Choose the driver
based on your
database (e.g. asyncpg
for PostgreSQL).

3. Database Configuration (database.py)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///test.db" # change for PostgreSQL, MySQL, etc.

engine = create_engine(
    DATABASE_URL, connect_args={"check_same_thread": False} # only for SQLite
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()
```

Change DATABASE_URL
for your database.
For PostgreSQL:
postgresql://user:pass@localhost/dbname

4. Define a Model (models.py)

```
from sqlalchemy import Column, Integer, String
from database import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    email = Column(String, unique=True, index=True)
```

Each model maps
to a database table
using SQLAlchemy ORM.

9. Supported Databases

- SQLite (default, good for development)
- PostgreSQL (recommended for production)
- MySQL
- Others (with appropriate drivers)

Just change the
DATABASE_URL
and install the
required driver.

5. Create Tables (main.py)

```
from fastapi import FastAPI
from database import engine, Base
from routers import user

app = FastAPI()

Base.metadata.create_all(bind=engine)
app.include_router(user.router)
```

This will create
the tables in the
database if they
don't exist.

6. Create Database Dependency

```
from database import SessionLocal
from sqlalchemy.orm import Session

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

We use a dependency
to get a database
session for each request.

7. Use in API Route (CRUD Example)

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from . import models, schemas
from database import get_db

router = APIRouter()

@router.post("/users", response_model=schemas.User)
def create_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
    db_user = models.User(name=user.name, email=user.email)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user
```

We use the database
session (db) in our
route to perform
CRUD operations.

8. Pydantic Schemas (schemas.py)

```
from pydantic import BaseModel

class UserBase(BaseModel):
    name: str
    email: str

class UserCreate(UserBase):
    pass

class User(UserBase):
    id: int
    class Config:
        orm_mode = True
```

Pydantic schemas
validate request data
and serialize response
data from SQLAlchemy
models.

10. Key Takeaways

- ✓ Use SQLAlchemy for database integration with FastAPI.
- ✓ Define models using SQLAlchemy ORM.
- ✓ Use a database session dependency in your routes.
- ✓ Create Pydantic schemas for request and response.
- ✓ Supports multiple databases (SQLite, PostgreSQL, MySQL, etc.).
- ✓ Simple, powerful and widely used in real world projects.

Database integration
makes your API
production ready
and useful in
real applications.

Learn
Build
Implement
Repeat!

@codedsoul_05

Same
Code
Bigger Dreams!

17. CRUD API + Repository Patterns

@codedsoul_05

Build
Real World
Projects
Step by Step!

"A clean separation of concerns makes your code scalable, testable and easy to maintain."

1. What is CRUD API ?

- CRUD stands for Create, Read, Update and Delete.
- It provides basic operations to manage resources (e.g. users, products, posts, etc).
- In FastAPI, we define these using path operations (GET, POST, PUT, DELETE).
- Used in almost every real world application.

2. Project Structure (using Repository Pattern)

```
app/
├── main.py
├── database.py
├── models/
│   └── user.py
├── schemas/
│   └── user.py
├── repositories/
│   └── user_repository.py
├── routers/
│   └── users.py
└── dependencies.py
```

Repository pattern
keeps your business
logic separate from
your API routes.

3. Database Setup (database.py)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///test.db" # change for production
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

Use environment
variables for DATABASE_URL
in production.

4. Model (models/user.py)

```
from sqlalchemy import Column, Integer, String, Boolean
from app.database import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, nullable=False, index=True)
    is_active = Column(Boolean, default=True)
```

This model maps
to the users table
in the database.

7. CRUD Routes (routers/users.py)

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import SessionLocal
from app.schemas.user import UserCreate, UserUpdate, UserResponse
from app.repositories.user_repository import UserRepository

router = APIRouter(prefix="/users", tags=["Users"])

def get_repo(db: Session = Depends(SessionLocal)):
    return UserRepository(db)

@router.get("/", response_model=list[UserResponse])
def get_users(repo: UserRepository = Depends(get_repo)):
    return repo.get_all()
```

Define clean API
routes and use the
repository to handle
database logic.

5. Schema (schemas/user.py)

```
from pydantic import BaseModel, EmailStr

class UserCreate(BaseModel):
    name: str
    email: EmailStr
    is_active: bool = True

class UserUpdate(BaseModel):
    name: str | None = None
    email: EmailStr | None = None
    is_active: bool | None = None

class UserResponse(BaseModel):
    id: int
    name: str
    email: EmailStr
    is_active: bool

class Config:
    orm_mode = True
```

Pydantic schemas
validate request data
and define the
response format.

6. Repository (repositories/user_repository.py)

```
from sqlalchemy.orm import Session
from app.models.user import User
from app.schemas.user import UserUpdate

class UserRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_all(self):
        return self.db.query(User).all()

    def get_by_id(self, user_id: int):
        return self.db.query(User).filter(User.id == user_id).first()

    def create(self, user_data):
        db_user = User(**user_data)
        self.db.add(db_user)
        self.db.commit()
        self.db.refresh(db_user)
        return db_user

    def update(self, user_id: int, data: UserUpdate):
        user = self.get_by_id(user_id)
        if not user:
            return None
        for key, value in data.dict(exclude_unset=True).items():
            setattr(user, key, value)
        self.db.commit()
        self.db.refresh(user)
        return user

    def delete(self, user_id: int):
        user = self.get_by_id(user_id)
        if not user:
            return None
        self.db.delete(user)
        self.db.commit()
        return True
```

Repository handles
database operations.
Routes stay clean
and focused on
request/response.

```
@router.get("/{user_id}", response_model=UserResponse)
def get_user(user_id: int, repo: UserRepository = Depends(get_repo)):
    user = repo.get_by_id(user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user

@router.post("/", response_model=UserResponse, status_code=201)
def create_user(user: UserCreate, repo: UserRepository = Depends(get_repo)):
    return repo.create(user.dict())

@router.put("/{user_id}", response_model=UserResponse)
def update_user(user_id: int, user: UserUpdate, repo: UserRepository = Depends(get_repo)):
    updated_user = repo.update(user_id, user)
    if not updated_user:
        raise HTTPException(status_code=404, detail="User not found")
    return updated_user

@router.delete("/{user_id}")
def delete_user(user_id: int, repo: UserRepository = Depends(get_repo)):
    success = repo.delete(user_id)
    if not success:
        raise HTTPException(status_code=404, detail="User not found")
    return {"message": "User deleted successfully"}
```

Full CRUD
implementation
using repository
pattern.

8. Key Takeaways

- ✓ CRUD APIs are the foundation of most real world applications.
- ✓ Repository pattern separates database logic from API routes.
- ✓ Makes your code modular, testable and maintainable.
- ✓ Works with multiple databases (SQLite, PostgreSQL, MySQL, etc).
- ✓ Follow this pattern in larger projects for clean architecture.

Same
Code
Bigger
Opportunities!

@codedsoul_05

Test
Build
Refactor
Ship Better APIs!

18. Testing + Pytest + TestClient

@codedsoul_05

Better
Tests
Happier
Developers!

"Testing gives you confidence to build and ship better APIs with fewer bugs."

1. Why Testing is Important ?

- Helps catch bugs early.
- Ensures your API works as expected.
- Makes your code more reliable and maintainable.
- Safe to refactor and add new features.
- Increases confidence while deploying to production.

2. Testing Tools in FastAPI

Tool	Purpose	Notes
pytest	Testing framework	Simple and powerful for Python.
TestClient	Test your FastAPI app	Simulates HTTP requests to your API.
httpx	Async HTTP client (optional)	Useful for async tests and external APIs.
pytest-asyncio	Run async test functions	Required for async tests.

FastAPI provides TestClient built on top of requests, so no need to run a separate server.

3. Install Required Packages

```
pip install pytest
pip install httpx
pip install pytest-asyncio # for async tests (optional)
```

Install these in your development environment.

4. Project Structure for Testing

```
app/
├── main.py
├── routers/
│   └── users.py
├── tests/
│   ├── test_main.py
│   └── test_users.py
└── pyproject.toml # or requirements.txt
```

Keep tests in a separate folder (tests/) to keep your project clean and organized.

5. Basic Test with TestClient

```
from fastapi.testclient import TestClient
from app.main import app
```

```
client = TestClient(app)
```

```
def test_read_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello from FastAPI"}
```

TestClient allows you to send HTTP requests to your app without running a real server.

6. Testing Path Parameters and Query Parameters

```
def test_read_user():
    response = client.get("/users/1")
    assert response.status_code == 200
    assert response.json() == {"id": 1, "name": "Nikhil"}

def test_search_users():
    response = client.get("/users", params={"q": "nikhil"})
    assert response.status_code == 200
    assert "nikhil" in response.json()["users"][0]["name"]
```

You can test both path and query parameters easily.

7. Testing Request Body (POST)

```
def test_create_user():
    data = {"name": "Nikhil", "email": "nikhil@example.com"}
    response = client.post("/users", json=data)
    assert response.status_code == 201
    assert response.json()["name"] == "Nikhil"
```

Use json=data to send request body in tests.

8. Testing Error Responses

```
def test_user_not_found():
    response = client.get("/users/999")
    assert response.status_code == 404
    assert response.json()["detail"] == "User not found"
```

Always test error cases too.

9. Testing with Database (Example)

```
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.user import User

def test_get_users(db: Session, client):
    # Add test data
    user = User(name="Test User", email="test@example.com")
    db.add(user)
    db.commit()

    response = client.get("/users")
    assert response.status_code == 200
    assert any(u["email"] == "test@example.com" for u in response.json()["users"])
```

Use a test database (e.g. SQLite) and dependencies override for get_db().

10. Running Tests

```
pytest -v
```

The -v flag shows detailed test results.

11. Key Takeaways

- ✓ Use pytest for testing FastAPI applications.
- ✓ Use TestClient to simulate HTTP requests.
- ✓ Test all major endpoints (GET, POST, PUT, DELETE).
- ✓ Test both success and error responses.
- ✓ Use a separate test database for database tests.
- ✓ Keep your tests organized inside a tests/ folder.
- ✓ Write tests early and run them regularly.

Good tests make your API production ready and easier to maintain.

Write Tests
Catch Bugs
Build Confidence!

@codedsoul_05

Test
Improve
Grow
Repeat!

Build
Package
Deploy
Scale
Repeat !

19. Docker + Deployment + Production

@coded soul_05

Code
Build
Deploy
Grow !

"From your local machine to the real world — Docker and proper deployment make your API reliable, scalable and always available."

1. Why Docker ?

- Packages your application with all dependencies.
- Works the same on any machine (no "it works on my machine" issues).
- Makes deployment easier and more reliable.
- Lightweight, fast and widely used in production.
- Helps in scaling and managing multiple services.

2. Dockerfile for FastAPI

```
# Use official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy and install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Expose port
EXPOSE 8000

# Run the application
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile tells Docker how to build your FastAPI application image.

3. Docker Compose (Optional)

```
version: '3.8'
services:
  app:
    build: .
    container_name: fastapi_app
    ports:
      - "8000:8000"
    environment:
      - ENV=production
    depends_on:
      - db
  db:
    image: postgres:15
    container_name: fastapi_db
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
      - POSTGRES_DB=fastapi
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Docker Compose helps you run multiple services (app, database, etc.) together.

4. Build and Run Commands

```
# Build the Docker image
docker build -t fastapi-app .

# Run the container
docker run -p 8000:8000 fastapi-app

# Using Docker Compose
docker-compose up --build
```

These commands build and run your FastAPI app locally using Docker.

5. Deploy on Cloud Platforms

- Popular options:
- Render (easy and beginner friendly)
- Railway (simple and fast)
- Fly.io (global deployment)
- AWS (EC2, ECS, Elastic Beanstalk)
- Google Cloud Run
- DigitalOcean App Platform
- Azure (App Service)

Choose a platform based on your needs (budget, scalability, features).

6. Deploy on Render (Example)

- Push your code to GitHub.
- Go to render.com and create a new Web Service.
- Connect your GitHub repository.
- Set build command: `pip install -r requirements.txt`
- Set start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- Add environment variables (if needed).
- Deploy and get your live URL.

Render automatically builds and deploys your Docker or source code.

7. Using Reverse Proxy (Nginx) in Production

```
server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Nginx acts as a reverse proxy and forwards requests to your FastAPI app.

8. Environment Variables

- Use .env file locally.
- Set environment variables in your deployment platform.
- Access them in FastAPI using `os.getenv()` or `pydantic`.

Keep sensitive data like database URLs and API keys in environment variables.

9. Production Best Practices

- Use a production grade server (Uvicorn with Gunicorn).
- Enable HTTPS (with SSL).
- Use environment variables for secrets.
- Set up logging and monitoring.
- Use a process manager (e.g. systemd, Docker, or cloud platform).
- Configure automatic restarts.
- Use a database for production (PostgreSQL, not SQLite).
- Set proper CORS and security headers.

Make your app secure, reliable and ready to handle real users.

10. Key Takeaways

- ✓ Docker packages your FastAPI app with all dependencies.
- ✓ Docker Compose helps run multiple services.
- ✓ You can deploy on platforms like Render, Railway, AWS, etc.
- ✓ Use Nginx as a reverse proxy in production.
- ✓ Always use environment variables for sensitive data.
- ✓ Follow production best practices for a secure and scalable API.

Containerize
Simplify
Deploy Faster !

@coded soul_05

Same
Code
Bigger
Impact !

Learn
Build
Test
Deploy
Repeat!

20. FastAPI Cheat Sheet + 100 Interview Questions

@coded soul_05

Master
the fundamentals.
Build
production APIs.

"FastAPI + your skills + consistent practice = Real world opportunities."

1. Setup & Create App

```
# Install FastAPI with standard dependencies
uv add "fastapi[standard]"
# Run development server
uv run fastapi dev

# main.py
from fastapi import FastAPI
app = FastAPI(title="My API", version="1.0.0")

@app.get("/")
def root():
    return {"message": "Hello from FastAPI"}
```

Modern way to run
FastAPI (from official
docs) using fastapi dev

Automatic docs
available at
/docs
/redoc
/openapi.json

100 Interview Questions

Q1 - Q10: Basics

1. What is FastAPI?
2. What are the main features?
3. How is it different from Flask?
4. Why is it called fast?
5. What is ASGI?
6. What is Uvicorn?
7. What are the advantages?
8. What are the disadvantages?
9. When to use FastAPI?
10. How to create a FastAPI app?

Q11 - Q20: Routing and HTTP

11. What are path operations?
12. How to define GET, POST, PUT, DELETE?
13. What is a path parameter?
14. What is a query parameter?
15. How to set route prefixes?
16. How to handle trailing slashes?
17. What is APIRouter?
18. How to include routers?
19. How to set tags and metadata?
20. How to handle versioning?

2. Path Operations

```
@app.get("/items/{item_id}")
def get_item(item_id: int, q: str = None):
    return {"item_id": item_id, "query": q}
```

3. Request Body (Pydantic)

```
from pydantic import BaseModel
class Item(BaseModel):
    name: str
    price: float
    description: str | None = None

@app.post("/items/")
def create_item(item: Item):
    return item
```

4. Query, Path, Headers, Cookies, Forms, Files

```
from fastapi import Query, Header, Cookie, Form, File, UploadFile
@app.get("/search")
def search(q: str = Query(...)):
    return {"q": q}
```

FastAPI supports
path, query, header,
cookie, form and
file parameters.

5. Response Model & Status Code

```
from fastapi import status
@app.get("/users/{id}",
        response_model=Item,
        status_code=status.HTTP_200_OK)
def get_user(id: int):
    return Item(name="Nikhil",
        price=0.0, description="Test")
```

6. Dependencies (Depends)

```
from fastapi import Depends
def get_db():
    # return db session
    ...

@app.get("/items/")
def read_items(db = Depends(get_db)):
    return {"items": []}
```

Use Depends
for dependency
injection.

7. Middleware & CORS

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Use CORS
for frontend
integration.

8. Exception Handling

```
from fastapi import HTTPException, Request
from fastapi.responses import JSONResponse
@app.exception_handler(Exception)
async def global_exception_handler(request: Request,
        exc: Exception):
    return JSONResponse(
        status_code=500,
        content={"detail": "Internal Server Error"})
```

9. Background Tasks

```
from fastapi import BackgroundTasks
@app.post("/send-email")
def send_email(background_tasks: BackgroundTasks):
    background_tasks.add_task(send_email_func)
    return {"message": "Email scheduled"}
```

10. Database (SQLAlchemy)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

# Use SQLAlchemy with Depends to get DB session.
```

Use SQLAlchemy
with Depends
to get DB session.

11. Authentication (OAuth2 + JWT)

```
from fastapi.security import OAuth2PasswordBearer
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/protected")
def protected_route(token: str = Depends(oauth2_scheme)):
    return {"token": token}
```

FastAPI has built-in
security tools like
OAuth2PasswordBearer.

12. Testing (Pytest + TestClient)

```
from fastapi.testclient import TestClient
from app.main import app
client = TestClient(app)

def test_root():
    response = client.get("/")
    assert response.status_code == 200
```

TestClient is based
on HTTPX and
allows testing
without running
a real server.

13. Deployment (Docker + Production)

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN uv pip install .
CMD ["uvicorn", "app.main:app",
    "--host", "0.0.0.0", "--port", "8000"]
```

Use a production
grade server (e.g.
Uvicorn with Gunicorn)
and enable HTTPS
with a reverse proxy
(Nginx).

14. Key Takeaways

- ✓ FastAPI is fast, easy to use and production ready.
- ✓ Automatic OpenAPI docs at /docs and /redoc.
- ✓ Use Pydantic for data validation.
- ✓ Use Depends for clean and modular code.
- ✓ Test your APIs and follow best deployment practices.

Build
Real World
Projects!

Keep Learning
Keep Building
@coded soul_05

Small Progress
Big Results!

@coded soul_05